Al-Aqsa University
Faculty of Computers and IT
Dept. of Info & Computer science

جامعة الأقصى
كلية الحاسبات وتكنولوجيا المعلومات
قسم علم الحاسوب والمعلومات

Software Engineering

Assignment 2: Answers Of Chapters (4,5,6,7)

Presented to: **Eng. Abdulfattah AL-Farra**

Student name: **Deema Mohammed AL-Maqadma**
Student number:**2320230766**

Semester: Second Semester 2024/2025

→Chapter4

1.What are the three phases of the XP Planning Game, and what happens in each phase?

The XP Planning Game consists of three phases:

- Exploration Phase:
  Identify potential system features by writing, estimating, and splitting user stories. This phase focuses on brainstorming and understanding what the system could do.

- Commitment Phase:
  Decide which subset of requirements to implement next. Stories are sorted by value and risk, velocity is set, and a set of stories is chosen for the release.

- Steering Phase:
  Guide development by re-estimating remaining stories, adjusting velocity, and updating the plan based on progress and feedback.

2. Describe the five levels of planning and explain how each level connects to the others.

Level	Description	Connection
Product Vision	High-level statement describing the system's goal and target audience.	Sets the overall direction and ensures all other plans align with the business objectives.
Product Roadmap	Broad plan of future releases, listing high-level functionality or themes.	Breaks down the vision into achievable milestones over time.
Release Plan	Defines the features and user stories for a specific release.	Translates roadmap themes into actionable deliverables for a release.

Iteration Plan	Detailed plan of tasks for a 1–3 week iteration.	Breaks down release features into smaller, manageable tasks.
Daily Standup	Daily meeting to discuss progress, plans, and blockers.	Ensures iteration tasks are on track and identifies issues early.

3. How does the Product Roadmap differ from the Release Plan in terms of content and purpose?

Aspect	Product Roadmap	Release Plan
Content	High-level themes or functionality for future releases.	Detailed list of user stories prioritized by value.
Purpose	Provides a long-term vision and direction.	Focuses on short-term deliverables for a specific release.
Timeframe	Covers multiple releases (months or years).	Covers a single release (1-3 months).
Audience	Stakeholders and customers.	Development team and customers.

4. What role does velocity play in iteration planning in XP, and how is it calculated?

- Role of Velocity:
  Velocity measures the amount of work (in story points or ideal time) completed in an iteration. It helps teams estimate how much work they can commit to in future iterations and ensures realistic planning.

- How It's Calculated:
  Velocity is the total of the estimates for all completed stories in an iteration. For example, if a team completes stories worth 40 story points in an iteration, their velocity is 40.

5. What are the responsibilities of the Business and the Development team during an Iteration Planning meeting?

Team	Responsibilities
Business Team	- Selects user stories for the iteration based on priority and value.
	- Provides acceptance criteria for each story.
	- Adjusts scope if necessary.
Development Team	- Breaks down stories into tasks.
	- Estimates the time required for each task.
	- Commits to completing a realistic amount of work based on velocity.

→ Chapter 5

1. What is the purpose of a UML Use Case Diagram, and how are Actors and Use Cases represented?

A UML Use Case Diagram is used to visualize the functional requirements of a system from the perspective of users or other systems (actors).

- Purpose:
  It captures user interactions with the system and shows what the system should do (use cases), not how it does it.

- Representation:
  - Actors: Represented as stick figures outside the system boundary; they can be users or other systems interacting with the system.
  - Use Cases: Shown as ovals within the system boundary; each oval represents a functional unit initiated by an actor.

2. Compare Use Cases and User Stories in terms of their level of detail and main objectives.

Feature	Use Cases	User Stories
Detail Level	High: Describes full interaction flow and steps	Low: Brief, informal statement of need
Language	Formal, structured	Everyday, user-centric
Objective	Provide detailed process model	Capture user needs for planning
Usage	System design and documentation	Agile planning and sprint estimation

Summary:
Use Cases offer comprehensive narratives of user-system interaction, while User Stories provide concise goals for rapid development.

3. What are CRC cards (Class–Responsibility–Collaborator), and how are they used to discover and define classes?

CRC Cards are a lightweight modeling tool used to define and analyze object-oriented classes in a system.

- Structure: Each card contains:
  - Class Name: The role or entity being modeled.
  - Responsibilities: What the class knows or does.
  - Collaborators: Other classes that help fulfill the responsibilities.

- Usage:
  CRC cards help teams brainstorm and refine class structures. By writing cards on index paper, designers discover appropriate roles and how objects work together—especially useful in early design phases of XP.

4. In a UML Class Diagram, what is the difference between Association, Aggregation, and Composition?

Relationship	Definition	Symbol
Association	A general link between classes (e.g., one object uses another)	Straight line
Aggregation	A "whole–part" relationship where parts can exist independently	Empty diamond
Composition	Stronger "whole–part" relationship where parts are destroyed with the whole	Filled diamond

Example:
- Association: A Student is linked to Instructor.
- Aggregation: A Team has Players, but players exist outside the team.
- Composition: A Window has a Scrollbar, and if the window is destroyed, so is the scrollbar.

5. How do Sequence Diagrams complement Class Diagrams in modeling system behavior over time?

- Class Diagrams show static structure: classes, attributes, and relationships.

- Sequence Diagrams show dynamic behavior over time:
  - How objects interact.
  - In what order messages are sent.
  - The lifelines and activation periods of objects.

Complementary Use:
Class diagrams define what exists; Sequence diagrams illustrate how those entities interact in real-time scenarios. Together, they offer a full picture—structure + behavior

➔ Chapter 6

1.Describe the steps of the XP developer's daily routine—Pair-up, Quick Design, Write Test, Code, Refactor, Integrate, Go Home—and explain the importance of each step.

- Pair-up:
  Developers work in pairs, sharing a workstation. This step ensures collaboration, knowledge sharing, and higher code quality.

- Quick Design:
  A brief discussion on how to solve the task. This step avoids over-engineering and focuses on simplicity.

- Write Test:
  Write a test case before coding to ensure the task's requirements are clear and testable. This step prevents bugs and ensures functionality.

- Code:
  Write just enough code to pass the test. This step focuses on delivering working functionality without unnecessary complexity.

- Refactor:
  Simplify and improve the code without changing its behavior. This step ensures maintainability and removes technical debt.
- Integrate:
  Add the completed code to the main codebase and verify it works with existing components. This step ensures smooth integration and avoids conflicts.

- Go Home:
  Developers leave work on time (40-hour week principle). This step promotes sustainable productivity and prevents burnout.

2. What are the benefits and drawbacks of Pair Programming according to the University of Utah study?

- Benefits:
  - Higher Code Quality: Pairs produce fewer defects and shorter code.
  - Improved Collaboration: Encourages communication and knowledge sharing.
  - Faster Problem Solving: Mistakes are caught early, and designs are better.
  - Increased Enjoyment: Developers find the process more engaging and fun.

- Drawbacks:
  - Skill Differences: Unequal skill levels can lead to one partner dominating.
  - Socializing Risks: Some pairs may lose focus and socialize instead of working.
  - Ego Clashes: Conflicts may arise if programmers push their own ideas.
  - Evaluation Challenges: Difficult to assess individual contributions.

3. Explain the concept of Collective Ownership and how it impacts code quality and team dynamics.

- Concept:
  Collective Ownership means all team members share responsibility for the entire codebase. Anyone can modify any part of the code at any time.

- Impact on Code Quality:
  - Encourages simplicity by preventing complex code from entering the system.
  - Improves quality as multiple developers review and refine the code.
  - Reduces project risk by spreading knowledge across the team.

- Impact on Team Dynamics:
  - Promotes collaboration and shared responsibility.
  - Reduces dependency on specific individuals.
- Requires developers to let go of personal attachment to their code

4. Why are Coding Standards necessary in an XP environment, and what key characteristics should they include?

- Necessity:
  Coding Standards ensure consistency across the codebase, making it easier for developers to understand and modify each other's work. They also prevent conflicts when pairs switch tasks.

- Key Characteristics:
  - Simplicity: Avoid unnecessary complexity and duplication.
  - Clarity: Emphasize communication and readability.
  - Team Agreement: Standards must be adopted voluntarily by the team.
  - Maintainability: Focus on long-term ease of updates and modifications.

5. Define Refactoring and list three code smells that signal its need.

- Definition:
  Refactoring is the process of improving code without changing its functionality. It simplifies the code, enhances readability, and reduces technical debt.

- Code Smells:
  1. Duplicated Code: Identical code appears in multiple places.
  2. Long Method: Methods are too lengthy and hard to understand.
  3. Large Class: A class has too many responsibilities or instance variables

➔ Chapter 7

1.What is the difference between Unit Tests and Acceptance Tests in XP, and what role does each play in ensuring software quality?

- Unit Tests:
  - Definition: Automated tests written by developers to test individual classes or small clusters of classes.
  - Role: Ensure that each unit of the code works as expected. They are written before the code and are part of the development process.
  - Scope: Focused on small, isolated pieces of functionality.

- Acceptance Tests:
  - Definition: Tests specified by the customer to validate that the system meets the requirements.
  - Role: Ensure that the overall system functions as intended and satisfies user stories. - Scope: Test larger blocks of functionality or the entire system.

Summary:
Unit Tests verify the correctness of individual components, while Acceptance Tests validate the system's behavior from the user's perspective.

2. Explain the Test-Driven Development (TDD) cycle and its benefits for improving reliability.

- TDD Cycle:
  1. Write a Test: Before writing any code, create a test for the desired functionality.
  2. Run the Test: The test will fail initially since the code hasn't been written yet.
  3. Write Code: Write the minimum amount of code required to pass the test.
  4. Run the Test Again: Ensure the test passes.
  5. Refactor: Improve the code while ensuring the test still passes.
  6. Repeat: Continue the cycle for each new feature or functionality.

- Benefits:
  - Reduces debugging time by catching errors early.
  - Increases confidence in the code's correctness.
  - Ensures new features work without breaking existing functionality.
  - Encourages better design and maintainability.

3. How are Test Fixtures (@Before, @After, @BeforeClass, @AfterClass) used in JUnit to manage setup and teardown?

- Test Fixtures: Provide a consistent environment for running tests by setting up and tearing down resources.

Annotation	Purpose
@Before	Runs before each test to initialize resources (e.g., create objects).
@After	Runs after each test to release resources (e.g., close files).
@BeforeClass	Runs once before all tests in the class (e.g., connect to a database).
@AfterClass	Runs once after all tests in the class (e.g., clean up shared resources).

4. What are the main JUnit assert methods, and how do assertions for floating-point values differ?

- Main Assert Methods:

Method	Purpose
`assertEquals()`	Checks equality between expected and actual
`assertTrue()`	Verifies that a condition is true
`assertFalse()`	Verifies that a condition is false
`assertNull()`	Confirms that an object is null
`assertNotNull()`	Confirms that an object is not null
`assertSame()`	Verifies that two references point to the same object
`assertNotSame()`	Verifies that two references do not point to the same object
`assertArrayEquals()`	Compares arrays element by element

It checks whether the absolute difference is within an acceptable margin, avoiding false failures due to rounding.

5. Compare White-Box Testing and Black-Box Testing in terms of test design and objectives.

Aspect	White-Box Testing	Black-Box Testing
Test Design	Based on internal code logic	Based on software specifications and functionality
Knowledge Needed	Requires access to source code	No code knowledge required
Focus	Verifying control flow, conditions, and logic paths	Validating outputs against inputs
Common Use Cases	Unit testing, path coverage, branch testing	System testing, acceptance testing

Summary:

White-box testing ensures the internal mechanics work correctly, while black-box testing ensures the system behaves as users expect from the outside.

" واخر دعواهم ان الحمد لله رب العالمين "